Modélisation et conception d'une BDD avec UML2

Formateur : Bastien Flanquart

Sommaire

Introduction *(1ère partie)*

- Qu’est-ce que UML2 ?
- À quoi ça sert ?

Introduction *(2ème partie)*

- Pourquoi modéliser ?
- Diagramme de classes

UML

- Les éléments UML essentiels
- Lire les multiplicités
- Association N-N en pratique

Exemples

- Diagramme UML Client / Commande / Produit
- Traduction en tables SQL (PK, FK)

INTRODUCTION

INTRODUCTION

Qu'est-ce que UML2 ?

Qu'est-ce que UML2 ?

UML est un **langage de modélisation** standard utilisé dans le monde entier pour **représenter visuellement la structure et le comportement d'un système informatique**.

- **UML:** Unified Modeling Language (Langage de Modélisation Unifié)
- **Langage visuel standardisé** pour décrire des systèmes informatiques
- **UML 2:** version améliorée avec **14 types de diagrammes**
- Utilisé pour la **conception orientée objet** et la **modélisation de bases de données**

À quoi ça sert ?

La modélisation permet de **traduire les besoins métier** en structures techniques claires, facilitant ainsi la communication entre développeurs, clients et gestionnaires de projet.

- **Communiquer :** comprendre et partager la structure d'un système
- **Concevoir :** réfléchir avant de coder ou créer des tables
- **Documenter :** garder une trace visuelle du projet
- **Pour les BDD :** modéliser les tables et leurs relations avant l'implémentation SQL

INTRODUCTION

Pourquoi modéliser ?

Pourquoi modéliser une base de données ?

Modéliser une base de données avant de l'implémenter permet d'**anticiper les problèmes de structure**, d'éviter la redondance des données et de **garantir l'intégrité des informations stockées**.

- **Éviter les erreurs :** identifier les entités et relations avant de créer les tables
- **Gagner du temps :** restructurer un diagramme est plus rapide que refaire du SQL
- **Normaliser :** respecter les formes normales et éviter la redondance
- **Vision globale :** comprendre toute la structure d'un coup d'œil

Le diagramme de classes UML

Le diagramme de classes est l'outil central pour concevoir une base de données relationnelle : il représente les **entités**, leurs **attributs** et les **relations** qui les lient.

Le diagramme le plus utilisé pour modéliser une base de données relationnelle.

- **Classe :** future table de la BD
- **Attribut :** future colonne (avec son type)
- **Association :** lien entre tables (clé étrangère ou table de jointure)

UML

UML

Les éléments UML essentiels

Les éléments UML essentiels

Pour modéliser une base de données avec UML, vous devez maîtriser trois éléments fondamentaux : **les classes (entités)**, **les attributs** (propriétés) et **les associations (relations)**.

Classe

Rectangle avec nom, attributs et types

```
Client
-------------
idClient : int
nom : String
email : String
```

Association

Trait reliant deux classes avec multiplicités

```
Client ------------- Commande
   1                 0..*
```

Lire les multiplicités

Les multiplicités indiquent **combien d'instances d'une classe peuvent être associées à une instance d'une autre classe** : elles définissent les **cardinalités des relations** (1-1, 1-N, N-N).

- **1 :** exactement un
- **0..1 :** zéro ou un (optionnel)
- **1..* :** un ou plusieurs
- **0..* (ou *) :** zéro, un ou plusieurs

- Exemple : Client 1 ——— 0..* Commande
  - Un client peut avoir **zéro**, **une** ou **plusieurs commandes**

Association N-N en pratique

Les relations **N-N** *(plusieurs-à-plusieurs (many-to-many))* ne peuvent pas être directement traduites en SQL : elles nécessitent une **table de jointure intermédiaire** qui stocke les associations et leurs attributs spécifiques.

Relation plusieurs-à-plusieurs (many-to-many)

- Une commande contient **plusieurs produits**
- **Un produit** peut apparaître dans plusieurs commandes
- **Solution :** créer une table de jointure intermédiaire

- Table **LigneCommande** avec **FK** vers **Commande** et **Produit** + attributs (quantité, prix)

Exemples

Exemples

Diagramme UML Client / Commande / Produit

Exemple : Client / Commande / Produit

Voici un cas pratique typique : un client peut passer plusieurs commandes, et chaque commande peut contenir plusieurs produits.
Ce schéma illustre une relation 1-N et une relation N-N.

```
Client
--------------
idClient : int
nom : String
prenom : String
email : String

Produit
--------------
idProduit : int
libelle : String
prix : float

Commande
-------------
idCommande : int
dateCommande : Date
idClient : int

LigneCommande
--------------
idCommande : int
idProduit : int
quantite : int
```

Associations

```
Client ------------- LigneCommande
   1                  0..*

LigneCommande ------------- Client
   0..*                      1
```

Traduction en tables SQL

Chaque classe UML devient une table SQL, chaque attribut devient une colonne, et les associations deviennent des clés étrangères ou des tables de jointure selon leur multiplicité.

CLIENT
PK : idClient • Colonnes : nom, prenom, email

COMMANDE
PK : idCommande • FK : idClient → CLIENT • Colonnes : dateCommande

PRODUIT
PK : idProduit • Colonnes : libelle, prix

LIGNE_COMMANDE (table de jointure)
PK : (idCommande, idProduit) • FK : idCommande → COMMANDE, idProduit → PRODUIT • Colonnes : quantite

Outils pour créer vos diagrammes UML

Outils pour créer vos diagrammes UML

De nombreux outils existent pour dessiner vos diagrammes UML : des solutions en ligne gratuites aux logiciels professionnels payants, chacun avec ses avantages selon vos besoins.

En ligne (gratuit)

- **draw.io (diagrams.net)**
- **Lucidchart**
- **PlantUML**

Logiciels pro

- **StarUML**
- **Entreprise Architect**
- **Visual Paradigm**

Modélisation et conception d'une BDD avec UML2

Formateur : Bastien Flanquart